

Plan de Saneamiento -- Dr. Bike Sydney

Repositorio: `github.com/Peredodiego2026/Dr.Bike-Sydney`
Stack: Vercel + Supabase + Stripe + Twilio + Resend + Anthropic (Claude)
Estado actual: Pre-lanzamiento (MVP funcional con deuda tecnica)
Ejecucion: Persona no tecnica usando Claude Code, sesion por sesion

Diagnostico Inicial

El proyecto es un PWA serverless para servicio de reparacion de bicicletas a domicilio en Sydney. Funciona, pero acumulo problemas por crecimiento rapido sin refactorizacion:

- ~4,900 lineas en `index.html` (monolitico HTML+CSS+JS)
 - 4 versiones distintas de la app mobile (codigo muerto)
 - Varios archivos duplicados (95-100% identicos)
 - RLS deshabilitado en Supabase
 - Auth client-side debil (admin y mecanico)
 - XSS en templates de email
 - API key de Google Maps hardcodeada expuesta
 - Consola de debug en produccion
 - Bugs en stripe-webhook y send-email
 - Codigo duplicado entre archivos
 - Dependencia listada pero no usada
-

Resumen de Sesiones

Sesion	Tema	Issues resueltos	Tiempo estimado
1	Seguridad critica	4	~30 min
2	Auth + Base de datos	3	~45 min
3	Limpieza de dead code	6	~20 min
4	Correccion de bugs	5	~30 min
5	Modularizacion del frontend	1 (grande)	~60 min
6	Produccion readiness	4	~40 min

Orden recomendado: Secuencial (1 -> 2 -> 3 -> 4 -> 5 -> 6).
Las sesiones 3 y 4 pueden intercambiarse.
Las sesiones 3 y 5 son las que mas reducen la carga cognitiva del codebase.

Sesion 1: Seguridad Critica

Objetivo: Eliminar vulnerabilidades expuestas y fugas de datos.

Issues a resolver

1. **Eruda debug console en produccion** -- `mobile_latest.html:2845` expone DOM, network, console a cualquier usuario
2. **XSS en templates de email** -- `api/send-email.js` y `api/send-invoice.js` interpolan datos de usuario sin sanitizar
3. **Artefactos de extension en admin** -- `admin.html:3497` tiene URLs `moz-extension://`
4. **Google Maps API key expuesta** -- `index.html:55` key hardcodeada sin restriccion documentada

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Ejecuta estos 4 arreglos de seguridad, uno por uno, haciendo commit por cada uno:

1. ELIMINAR ERUDA DE PRODUCCION. En `mobile_latest.html`, busca cerca del final del archivo las lineas que cargan Eruda (`script` con `eruda.js` y `eruda.init()`). Comentalas o eliminalas.
Esto es una consola de debug que filtra datos a cualquier usuario.
2. SANITIZAR EMAILS CONTRA XSS. En `api/send-email.js` y `api/send-invoice.js`, todas las variables que vienen del request body (`name`, `clientName`, `service`, `address`, `mechNotes`, etc.) se interpolan directamente en HTML sin escapar. Importa la funcion `sanitize` de `_security.js` y aplicala a cada variable antes de interpolarla en el template HTML. No sanitizar datos que ya vienen formateados (como fechas o montos).
3. LIMPIAR ADMIN.HTML. Busca y elimina cualquier referencia a `moz-extension://` o URLs de extension de navegador que hayan quedado en el archivo. Son artefactos de guardado desde navegador.
4. DOCUMENTAR RESTRICCION DE GOOGLE MAPS KEY. La key esta hardcodeada en `index.html`. No la quites del codigo (es una key de frontend, debe ir ahi), pero:
 - Agrega un comentario HTML arriba de la linea del script de Google Maps que diga: "IMPORTANTE: Restringe esta API key en Google Cloud Console -> APIs & Services -> Credentials -> API Key -> HTTP referrers -> Agrega tu dominio de Vercel"
 - Verifica que `.env.example` tenga la variable `GOOGLE_MAPS_KEY` documentada

Verificacion final: Ejecuta `git diff --stat` para confirmar que los 4 cambios estan hechos.

Verificacion

- ☐ `git diff --stat` muestra cambios en 4-5 archivos
- ☐ Eruda ya no se carga en `mobile_latest.html`
- ☐ Los templates de email usan `sanitize()`
- ☐ No hay rastros de `moz-extension://` en `admin.html`
- ☐ El comentario de Google Maps key existe en `index.html`

Sesion 2: Auth y Base de Datos

Objetivo: Implementar autenticacion real y habilitar Row Level Security.

Issues a resolver

1. **RLS deshabilitado en bookings** -- tabla bookings sin politicas de acceso
2. **Auth de admin client-side** -- password chequeado solo en el navegador
3. **PIN de mecanico debil** -- input visible, sin verificacion contra BD

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Vamos a arreglar la autenticacion y seguridad de base de datos.

1. **HABILITAR ROW LEVEL SECURITY EN BOOKINGS.** Lee scripts/add-bookings-rls.sql actual. Diseña y ejecuta politicas RLS para la tabla bookings en Supabase:
 - Clientes autenticados: pueden ver/crear sus propios bookings (usando auth.uid())
 - Mecanicos autenticados: pueden ver/actualizar bookings de su van asignado
 - Admin (service_role): puede ver/modificar todos los bookingsActualiza el archivo SQL con las politicas finales (sin comentar) y documenta los pasos manuales para aplicarlas en el dashboard de Supabase (SQL Editor).
2. **CREAR SISTEMA DE AUTH PARA ADMIN.** Actualmente admin.html tiene un password en cliente sin proteccion real. Implementa:
 - Un endpoint api/admin-auth.js que use Supabase Auth (signInWithPassword) para validar email + password contra la tabla auth.users de Supabase
 - El endpoint devuelve el access_token de Supabase
 - Actualiza admin.html para que el login use ese endpoint y almacene el token en sessionStorage
 - Documenta en un comentario que el admin debe crearse via Dashboard de Supabase -> Authentication -> Users -> Add User
3. **MEJORAR PIN DE MECANICO.** En mechanic.html:
 - Cambia el input de PIN a type="password"
 - El PIN debe verificarse contra la base de datos. Si la tabla vans tiene un campo pin, usalo. Si no, crea un endpoint api/mechanic-auth.js que verifique el PIN contra la tabla vans en Supabase.
 - La verificacion del PIN debe hacerse via fetch al endpoint, no solo en el cliente

Verificacion: Confirma que los nuevos endpoints de auth existen, que el SQL de RLS esta listo, y que los HTML usan los flujos de auth nuevos.

Verificacion

- ☐ api/admin-auth.js existe y funciona
- ☐ api/mechanic-auth.js existe y funciona
- ☐ scripts/add-bookings-rls.sql tiene politicas RLS activas (sin comentar)
- ☐ admin.html usa el endpoint de auth con Supabase
- ☐ mechanic.html usa input type="password" y endpoint de verificacion

Sesion 3: Limpieza de Codigo Muerto

Objetivo: Eliminar duplicados, archivos obsoletos y mockups no funcionales.

Issues a resolver

1. **4 versiones de app mobile** (mobile.html, v2, v3, latest)
2. **Duplicado applepay-test.html** identico a applepay.html
3. **index.html vs index-redesign.html** (95% identicos)
4. **admin.html.bak** en raiz de despliegue
5. **Mockups no funcionales** (payments.html, notifications.html)
6. **robots.txt y sitemap.xml** desactualizados

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Vamos a hacer limpieza agresiva de archivos que no se usan. El proyecto esta en pre-lanzamiento, no hay riesgo de romper produccion.

1. CONSOLIDAR APPS MOBILE. Hay 4 archivos: mobile.html, mobile_v2.html, mobile_v3.html, mobile_latest.html.
 - Deja solo mobile.html, renombrando mobile_latest.html -> mobile.html (sobrescribe el original)
 - Elimina mobile_v2.html y mobile_v3.html
 - Si hay referencias a estos archivos en otros lugares (manifest.json, robots.txt, sitemap.xml, sw.js, vercel.json), actualizalas
2. ELIMINAR DUPLICADOS.
 - Elimina applepay-test.html (es identico a applepay.html)
 - Elimina admin.html.bak
3. CONSOLIDAR INDEX.
 - Compara index.html con index-redesign.html
 - PREGUNTAME cual conservar: si el redesign es el diseño final -> index-redesign.html renombrado a index.html. Si el original es el bueno -> elimina index-redesign.html.
 - [NOTA PARA QUIEN EJECUTA: Responde cual conservar cuando Claude Code pregunte]
4. LIMPIAR MOCKUPS NO FUNCIONALES.
 - payments.html tiene referencias a archivos locales como "Dr.%20Bike%20%E2%80%94%20Payments_files/css2.css". Es un mockup guardado desde navegador, no funcional.
 - notifications.html tiene el mismo problema.
 - Mueve ambos a docs/mockups/ para referencia de diseño futuro
5. ACTUALIZAR SEO FILES.
 - robots.txt: reflejar solo los archivos HTML que realmente existen
 - sitemap.xml: actualizar con las URLs reales
 - .gitignore: agregar la linea *.bak

Verificacion: git status debe mostrar solo los archivos que quedan. No deben quedar duplicados ni archivos .bak en raiz.

Verificacion

- ☐ Solo existe un mobile.html (el contenido de mobile_latest)
- ☐ No existe applepay-test.html

- ☐ No existe `admin.html.bak`
 - ☐ No existe duplicado de index (solo uno queda)
 - ☐ `docs/mockups/` contiene `payments.html` y `notifications.html`
 - ☐ `robots.txt` y `sitemap.xml` reflejan los archivos reales
 - ☐ `.gitignore` incluye `*.bak`
-

Sesion 4: Correccion de Bugs

Objetivo: Arreglar bugs funcionales, eliminar codigo duplicado entre archivos JS, limpiar dependencias.

Issues a resolver

1. **Bug en stripe-webhook** -- `invoice.payment_failed` usa `membership` en vez de `membership_status`
2. **Bug en send-email -- template referral_success** -- variables fuera de scope
3. **Funcion normalizeAUPhone duplicada** -- identica en `send-sms.js` y `send-whatsapp.js`
4. **Config fragil de WhatsApp** -- numero guardado en tabla `van_zones` con hack
5. **Dependencia @anthropic-ai/sdk sin uso** -- en `package.json` pero no importada

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Corrige estos 5 bugs, uno por uno, con commit por cada uno:

1. BUG EN STRIPE-WEBHOOK. En `api/stripe-webhook.js`, en el handler de `invoice.payment_failed`, busca donde dice `membership` (como campo de la tabla `profiles`). En el resto del codigo ese campo se llama `membership_status`. Cambialo por `membership_status` para que sea consistente y el update realmente funcione.
2. BUG EN SEND-EMAIL - TEMPLATE REFERRAL_SUCCESS. En `api/send-email.js`, el template `referral_success` usa las variables `friendName` y `referralCount` pero nunca se definen (estan fuera de scope, siempre son `undefined`). Extrae `friendName` y `referralCount` de `req.body` al inicio del handler, igual que las demas variables. Si no vienen en el body, que tengan valores por defecto razonables.
3. DUPLICACION DE `normalizeAUPhone`. Esta funcion aparece identica en `api/send-sms.js` y `api/send-whatsapp.js`. Muevela a `api/_security.js`, exportala como `export function normalizeAUPhone`, e importala desde los otros dos archivos. Borra las copias duplicadas locales. Verifica que los imports usen la ruta correcta (probablemente `../_security.js`).
4. CONFIG FRAGIL DE WHATSAPP. En `api/send-whatsapp.js`, el numero de WhatsApp business (from) se lee de la tabla `van_zones` con `van_number=0`, `suburb='__whatsapp__'`. Esto es un hack fragil. Cambialo para que:
 - Primero intente leer de variable de entorno `TWILIO_WHATSAPP_FROM`
 - Si no existe, caiga al fallback actual (tabla `van_zones`)
 - Actualiza `.env.example` agregando `TWILIO_WHATSAPP_FROM`
5. DEPENDENCIA SIN USO. `@anthropic-ai/sdk` esta en `package.json` como dependencia pero `api/chat.js` y `api/diagnose-bike.js` usan `fetch` directo a la API de Anthropic, no el SDK. Elimina `@anthropic-ai/sdk` de `package.json` (de la seccion `dependencies`) y ejecuta

npm install para regenerar package-lock.json.

Verificacion: Lee cada archivo modificado para confirmar que no hay errores de sintaxis obvios. Si hay package.json, confirma que ya no lista @anthropic-ai/sdk.

Verificacion

- ☐ stripe-webhook.js usa membership_status consistentemente
- ☐ send-email.js template referral_success extrae variables de req.body
- ☐ _security.js tiene la funcion normalizeAUPhone exportada
- ☐ send-sms.js y send-whatsapp.js importan de _security.js
- ☐ send-whatsapp.js usa TWILIO_WHATSAPP_FROM como fuente primaria
- ☐ .env.example incluye TWILIO_WHATSAPP_FROM
- ☐ package.json ya no lista @anthropic-ai/sdk
- ☐ package-lock.json fue regenerado

Sesion 5: Modularizacion del Frontend

Objetivo: Separar CSS, JS y HTML del monolito de 4,900 lineas.

Issues a resolver

1. **index.html monolito** -- 4,900 lineas con HTML, CSS, y JS mezclados
2. **Diseño inconsistente** -- landing.html usa sistema de CSS distinto a index.html

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Modulariza el frontend. El archivo principal es index.html (~4900 lineas con HTML, CSS, y JS mezclados). Procede en este orden:

1. CREAR DIRECTORIOS. Crea los directorios css/ y js/ en la raiz del proyecto si no existen.
2. EXTRAER CSS DE INDEX.HTML.
 - Encuentra todos los bloques <style>...</style> en index.html
 - Mueve su contenido a un archivo css/main.css
 - Reemplaza los bloques <style> por <link rel="stylesheet" href="css/main.css">
 - Si hay estilos criticos para above-the-fold (primeras ~50 lineas que afectan el contenido visible sin scroll), deja un bloque <style> minimo inline para evitar flash of unstyled content
 - NOTA: Si index.html fue reemplazado por index-redesign.html en la sesion 3, trabaja con el archivo que quedo como index.html
3. EXTRAER JS DE INDEX.HTML.
 - Encuentra todos los bloques <script> inline (NO los que cargan CDN externos como Stripe, Supabase, Google Maps -- esos se quedan)
 - Mueve el contenido a dos archivos separados:
 - js/stripe.js: solo la logica de Stripe Elements, Payment Request, webhook handling
 - js/app.js: el resto de la logica (booking flow, UI, Supabase queries, etc.)
 - Reemplaza con <script type="module" src="js/app.js"></script> y similar para stripe

- Asegura que las variables globales que necesitan otros scripts sigan siendo accesibles

4. VERIFICAR QUE NO SE ROMPA.

- Revisa que los event listeners (DOMContentLoaded, onclick, addEventListener) sigan referenciando elementos que existen en el DOM al momento de ejecucion del script
- Si hay scripts inline que dependen de variables de otros scripts, usa el orden correcto de carga o type="module" con imports/exports

5. REPETIR PARA OTROS HTML GRANDES.

- Si admin.html (>3400 lineas) tambien es un monolito, repite el proceso (css/admin.css, js/admin.js)
- Si mechanic.html es grande, haz lo mismo

6. (OPCIONAL) UNIFICAR DISEÑO.

- landing.html usa un sistema de CSS distinto a index.html con sus propias variables
- Si ambos son parte del mismo producto, extrae variables comunes (colores, tipografia, espaciado) a un css/variables.css compartido
- Actualiza ambos HTML para usar el archivo compartido

Verificacion final: Confirma que index.html ahora es sustancialmente mas corto (idealmente <1000 lineas), principalmente HTML estructural con <link> y <script src>. Los archivos css/main.css, js/app.js y js/stripe.js deben existir y tener contenido.

Verificacion

- ☐ Existen `css/main.css` , `js/app.js` , `js/stripe.js`
- ☐ `index.html` es significativamente mas corto (< 1000 lineas ideal)
- ☐ Los `<link>` y `<script src>` apuntan a rutas correctas
- ☐ `admin.html` y `mechanic.html` tambien modularizados si eran grandes
- ☐ `css/variables.css` existe si se unifico diseno con landing

Sesion 6: Produccion Readiness

Objetivo: Dejar el proyecto listo para deploy productivo.

Issues a resolver

1. **Service worker borra todo el cache** en cada activacion
2. **Rate limiting en memoria** -- no persiste entre instancias serverless
3. **Apple Pay / Google Pay** -- spec disenado pero no implementado
4. **Falta health check y documentacion de deploy**

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Ultima sesion - dejamos el proyecto listo para produccion:

1. SERVICE WORKER - LIMPIEZA SELECTIVA. En sw.js, el evento activate actualmente borra TODOS los caches con `keys.map(k => caches.delete(k))`. Esto es demasiado agresivo: cada actualizacion obliga al usuario a re-descargar todo.

- Cambia el handler de activate para que SOLO borre los caches que NO sean la version actual (es decir, todo lo que no sea CACHE_STATIC ni CACHE_PAGES)
 - Mantén el resto del comportamiento (skipWaiting, claim)
2. RATE LIMITING CON RESPALDO COMPARTIDO. El rate limiting en `api/_security.js` es en memoria (Map), lo que significa que no comparte estado entre instancias de Vercel Functions. Agrega soporte para Upstash Redis como backend:
- Crea un helper en `_security.js` que detecte si las variables de entorno `UPSTASH_REDIS_URL` y `UPSTASH_REDIS_TOKEN` existen
 - Si existen, usa Upstash Redis (con fetch HTTP a la REST API de Upstash) para almacenar los contadores de rate limit
 - Si no existen, usa el Map en memoria actual como fallback
 - Documenta en `.env.example` las dos variables nuevas
 - NOTA: No instales paquetes nuevos. Upstash Redis tiene REST API, usa `fetch()`
3. APPLE PAY / GOOGLE PAY. Lee el spec en `docs/superpowers/specs/2026-05-30-apple-google-pay-design.md` e implementa los cambios que describe:
- En `api/create-subscription.js`: quita o comenta la línea `payment_method_types: ['card']` para que Stripe Checkout muestre automáticamente Apple Pay y Google Pay como opciones
 - En `index.html`: agrega el Payment Request Button de Stripe en el modal de pago, siguiendo el diseño del spec
 - Verifica que `.well-known/apple-developer-merchantid-domain-association` existe (ya debería estar)
4. HEALTH CHECK Y DOCUMENTACION DE DEPLOY.
- Crea un endpoint `api/health.js` que:
 - Devuelva JSON: `{ status: "ok", timestamp: new Date().toISOString() }`
 - Verifique conexión básica con Supabase (opcional, si `SUPABASE_URL` existe)
 - Responda en `<100ms`
 - Crea un archivo `DEPLOY.md` con:
 - Lista completa de variables de entorno requeridas (basado en `.env.example`)
 - Pasos para deployar en Vercel (conectar repo, configurar env vars, dominios)
 - URL del health check para monitoreo
 - Nota sobre restringir Google Maps API key
 - Nota sobre crear admin user en Supabase Auth dashboard

Verificación final:

- Recuenta los archivos del proyecto (`ls -la` o `dir`)
- Confirma que no hay `.bak`, no hay duplicados obvios
- Confirma que los endpoints de auth existen
- Confirma que `api/health.js` existe y es funcional
- Confirma que `DEPLOY.md` está completo
- Confirma que `.env.example` está actualizado

Verificación

- ☐ `sw.js` solo borra caches viejos, no los actuales
- ☐ `_security.js` tiene soporte de rate limiting con Upstash Redis (opcional)
- ☐ `.env.example` incluye `UPSTASH_REDIS_URL` y `UPSTASH_REDIS_TOKEN`

- ☐ `api/create-subscription.js` ya no restringe a solo `card`
 - ☐ Payment Request Button implementado en `index.html`
 - ☐ `api/health.js` existe y responde `{ status: "ok" }`
 - ☐ `DEPLOY.md` existe con todas las variables y pasos
 - ☐ Proyecto listo para conectar a Vercel y lanzar
-

Checklist Global de Cierre

Al finalizar las 6 sesiones, verifica:

- ☐ No hay archivos `.bak` en el proyecto
 - ☐ No hay archivos duplicados (>95% identicos)
 - ☐ Solo existe una version de cada pagina (index, mobile, admin, mechanic, landing)
 - ☐ Todos los HTML grandes estan modularizados (CSS y JS en archivos separados)
 - ☐ RLS esta habilitado y configurado en Supabase
 - ☐ Auth de admin y mecanico usa endpoints server-side, no solo cliente
 - ☐ Templates de email sanitizan datos de usuario (XSS)
 - ☐ No hay API keys de Google Maps sin documentacion de restriccion
 - ☐ No hay consolas de debug en produccion
 - ☐ Rate limiting funciona a nivel serverless (con o sin Redis)
 - ☐ Service worker no borra caches activos
 - ☐ Apple Pay / Google Pay estan habilitados
 - ☐ Existe health check y documentacion de deploy
 - ☐ `.env.example` esta completo y actualizado
 - ☐ `.gitignore` incluye `*.bak`
 - ☐ `package.json` no tiene dependencias sin usar
-

Notas para el Ejecutor (persona no tecnica)

1. **Antes de cada sesion:** Abre Claude Code en el directorio del proyecto clonado
2. **Durante la sesion:** Copia y pega el prompt completo de la sesion. Claude Code hara los cambios y te pedira confirmacion para acciones destructivas (borrar archivos, etc.)
3. **Commits:** Cada arreglo se commitea por separado. Si algo sale mal, puedes revertir el ultimo commit con `git revert HEAD`
4. **Branch:** Trabaja en una branch separada: `git checkout -b saneamiento-prod` antes de empezar
5. **Al final:** Haz `git push` para subir todo al repo remoto
6. **Supabase:** Las politicas RLS de la Sesion 2 requieren ejecutar SQL manualmente en el dashboard de Supabase. Claude Code te dara las instrucciones exactas.